Facultad de Ciencias UNAM

Practica05 de ICC



Autor:
Dorantes Torres Alexa

Número de cuenta:
324320356

23 de octubredel 2026

Parte 1: Calculando Pi a dardazos

Actividad 1.1: Un número fijo de dardos

Comenzaré con lo de uname -a, lo puse hasta el final perdón es que se me olvidó tomerle la captura, espero no afecte

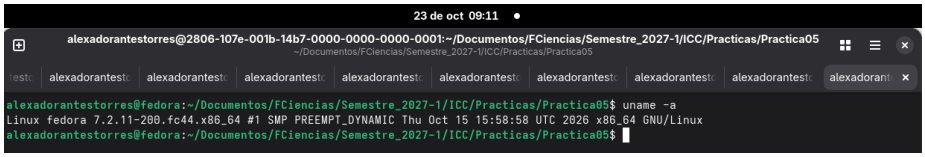
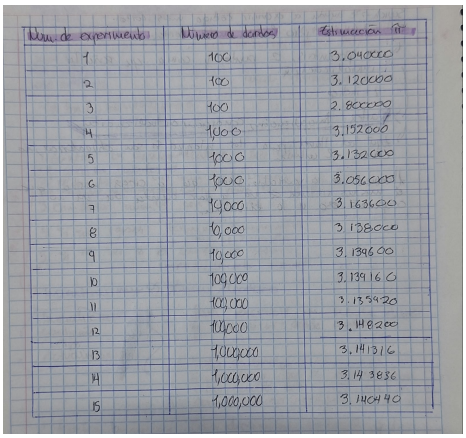


Figure 1: Comprobación uname

Prosiguiendo ahora si con lo que es del primer punto, se pide que en un cuarto de cuadrado y de círculo caigan muchos puntos (dardos), logrando que algúnos caigan dentro del cuarto de círculo y otros fuera de este mismo, como en la parte sobrante del cuadrado, entonces lo que hice como consiguiente fué crear mi directorio EstimacionPi.java y declaré ciertas variables, elegí como totalDeDardos 100, dardosDentroDelCirculo 0 porque como aún no se hace nada pues tiene que estar vacío y con numeroDeTiros lo mismo en 0.

En el paso 3 usé un ciclo while y una variable que funcionó como contador de los dardos lanzados, consiguiendo que se lanzaran de manera aleatoria, para esto como condición en mi while puse que el numeroDeTiros debe ser menos que el numeroDeDardos, entonces prosiguiendo declaré de nuevo otras dos variables en doubles, les puse x y a la otra y, esto porque indicarían como el tipo de coordenadas en parte de cómo se pondrían en un plano cartesiano los dardos que dan como resultado, y ocupé el Math.random que se nos enseñó en laboratorio y prosiguiendo ocupé un if con su debida condición de x por x t y por y menor o igual que 1.0, cotinuando a dardosDentroDelCírculo le puse ++, y a numeroDeTiros de igual manera ++, para que continue el ciclo que se pidió en la práctica; para el punto 4 tuve que aproximar pi, lograr que se imprimieran los números de dardos umero, la cantidad que cayó dentro del círculo y la aproximación de pi con 6 decimales contemplados, entonces para estimación de pi la verdad investigue porque no tenía ni idea y lo que encontré es que se podía hacer un casting, cosa que no había contemplado porque ni recordaba que existía sinceramente y esto fué para que los valores al dividirse no dieran un 0 en vez de lo que realmente ocupaba y se perdieran los datos, luego ya términando coloqué el cómo quería que cada cosa se imprimiera y me diera de igual manera los datos a los que deseaba llegar; para el punto 5 ejecuté mi programa 3 veces con los valores 100, 1,000, 10,000, 100,000 y 1,000,000, ahora mostraré la tabla como resultado.



Núm. de experimento	Número de dardos	Estimación Pi
1	100	3.040000
2	100	3.120000
3	100	2.800000
4	1,000	3.152000
5	1000	3.132000
6	1,000	3.056000
7	10,000	3.163600
8	10,000	3.138000
9	10,000	3.139600
10	100,000	3.139160
11	100,000	3.135920
12	100,000	3.140200
13	1,000,000	3.141316
14	1,000,000	3.143836
15	1,000,000	3.140440

Figure 2: Tabla de resultados de las ejecucuiones de mi programa

```
22 de oct 17:59 ●
alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Practica05
~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Practica05
alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documentos/ alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documen
alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Practica05$ javac -d bi
n src/EstimacionPi.java
alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Practica05$ java -cp bi
n EstimacionPi
Total de dardos lanzados:100
Dardos dentro del círculo:76
Estimación de Pi: 3.040000
alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Practica05$ javac -d bi
n src/EstimacionPi.java
alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Practica05$ java -cp bi
n EstimacionPi
Total de dardos lanzados:100
Dardos dentro del círculo:78
Estimación de Pi: 3.120000
alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Practica05$ javac -d bi
n src/EstimacionPi.java
alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Practica05$ java -cp bi
n EstimacionPi
Total de dardos lanzados:100
Dardos dentro del círculo:70
Estimación de Pi: 2.800000
alexadorantestorres@2806-107e-001b-14b7-0000-0000-0000-0001:~/Documentos/FCiencias/Semestre_2027-1/ICC/Practicas/Practica05$
```

Figure 3: Mis tres primeras ejecuciones con 100

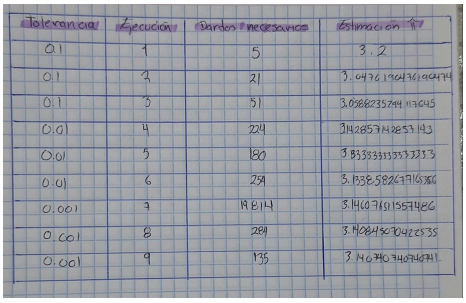
Pude observar que mientras el número de dardos subía, la estimación de pi se acercaba más a pi y dan ejecucuiones distintas porque en el programa como ya comenté se "pidió" que así fuera.

Actividad 1.2 - Repetir hasta llegar a la precisión deseada

En este segundo ejercicio el objetivo era cambiar la forma en que deteníamos la prueba. En el programa anterior simplemente le decíamos a Java que lanzara dardos y esperar a ver qué salía. Pero aquí no sabía anteriormente cuántos dardos se iban a necesitar para lograr el resultado requerido. La idea era fijar una

tolerancia, y hacer que el programa siguiera aventando dardos hasta que nuestro cálculo de pi se acercara lo suficiente al valor real de Math.PI; entonces para armar el código en PrecisionPi.java, primero definí la variable tolerancia con el valor que iba a probar (por ejemplo 0.1 como mi primer caso), luego creé los contadores dardos para saber cuántos lanzamientos llevábamos y dardosDentro para guardar solo los tiros que cayeron dentro del cuarto de círculo, de igual manera inicialicé una variable llamada errorAbsoluto en 1.0, este valor se lo puse siguiendo masomenos lo que le entendí al profe para que mi ciclo diera inicio, como consecuente comencé a usar while con la condición errorAbsoluto mayor o igual a tolerancia, lo cual indica que siga tirando dardos en pocas palabras; adentro del ciclo, en cada vuelta sumamos un dardo, generamos dos posiciones aleatorias como lo que ya había mensionado de las coordenadas x y la otra y, con Math.random y revisamos la fórmula que se nos había dado anteriormente ($x^2y^2 <= a1.0$) para ver si cayó dentro, luego en ese momento recalculamos la estimación de pi dividiendo los aciertos entre los dardos totales y multiplicando por 4, inmediato se saca la diferencia contra Math.PI usando Math.abs para actualizar nuestro errorAbsoluto; al esa diferencia por fin volverse más pequeña que la tolerancia, la condición del while se rompe y el programa se detiene, imprimiendo cuántos tiros necesitó y el resultado final.

Lo que pude observar en mis resultados de la tabla que realicé fué que por ejemplo en 0.1 me daba súper poquitos dardos, por ejemplo agarraba por lo regular dos primeras cifras después del punto y luego los ceros en contraria de 0.01 que en este caso incrementaba a 4 y en 0.001 que ocupaba en su mayoría los 6 decimales completos y encima lo que me dí cuenta que la verdad no sé si esté bien, espero que si, es que el valor poco a poco se iba aproximando a pi y al final terminó digamos en todos mis valores con 3.14...., por lo tanto pienso que el promedio si representa bien lo que "sucede" y considero que las diferencias entre el punto 1.1 y 1.2 de la practica son que en el 1.1 el programa lanza todos los dardos de un jalón y permite que las probabilidades medio se estabilicen, mientras que en el 1.2 el código empleado revisa el error dardo por dardo, creo que está raro pero es que en mi cabeza si tiene coherencia, en mi cabeza sería como un desfacé, obvio con otro tipo de palabra pero en si es esa mi interpretación.



Tolerancia	Ejecución	Dardos necesarios	Estimación en PI
0.1	1	5	3.2
0.1	2	21	3.047619047619047619
0.1	3	51	3.058823529411764645
0.01	4	224	3.142857142857143
0.01	5	180	3.133333333333333
0.01	6	259	3.1388.5882671916386
0.001	7	18114	3.1410761915574826
0.001	8	289	3.1408450704225535
0.001	9	135	3.1467407407407411

Figure 4: Resultados ejecución de estimación de pi

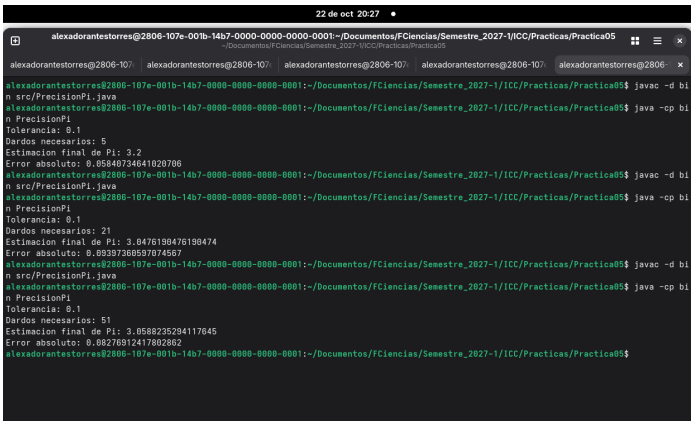


Figure 5: Estimación pi

Actividad 2.1 - Verificar los datos y el tipo de cliente

Para desarrollar esta actividad, el punto de este espacio de la práctica es la preparación de datos, la validación de la información, los cálculos de las tasas y la simulación del interés compuesto. Primero, declaré las variables iniciales con los datos que nos asignó la práctica. Definí el nombre del cliente como Robbie Valentino, el monto del préstamo en 12,899, el tiempo a 21 meses y la categoría del cliente con el carácter 'E' para Estándar, fijando además la tasa base inicial en 15.0%. Antes de realizar cualquier operación matemática, el programa ejecuta un bloque de validación usando instrucciones if. Aquí se revisa que el dinero prestado y los meses sean valores estrictamente mayores a cero, y que la letra del cliente sea únicamente 'P', 'E' o 'R'. Si alguna de estas condiciones no se cumple porque se ingresó un dato inválido, el programa imprime un mensaje indicando el error y ejecuta un return para detener la corrida en ese mismo momento. Una vez confirmados los datos, se calcula el ajuste de la tasa mediante un bloque if-else if. Si el cliente fuera Preferente ('P'), se le descuenta un 2.0 por ciento a la tasa, si fuera de Riesgo ('R'), se le incrementa un 3.0 por ciento, y como en este caso fue Estándar ('E'), la tasa se quedó intacta en 15.0 por ciento. Para obtener el interés simple, convertí el tiempo dividiendo los meses entre 12.0 para tener el dato en años con decimales y apliqué la fórmula directa. Para la parte del interés compuesto, saqué primero la tasa mensual dividiendo la tasa anual entre 1200 y programé un ciclo for que va ejecutándose desde el mes 1 hasta el mes 21. En cada vuelta del ciclo, el programa calcula el saldo acumulado multiplicando el monto anterior por (1.0 + tasaMensual) e imprime la línea correspondiente en la terminal para mostrar el crecimiento de la deuda mes a mes. Finalmente, para comprobar la exactitud matemática del proceso, utilicé la función Math.pow() con la fórmula directa del interés compuesto. Como siguiente puse a prueba lo del punto 4 y en estos vi que con los char E, P y R es que el valor cambiaba al imprimir los resultados, la verdad como que no había pensado que iba a pasar eso pero de primera instancia

me quedé como pensando que algo lo había hecho mal, viendo que no ya me quedé más tranquila y proseguí con ahora si los errores que se pedían, en el primero cambié precioClinte1 a 0, y evidentemete no se podría hacer eso porque si se supone que le debe y por eso es el problema, continuando cambié el char E a e y arrojo mal porque esta variable no existe entonces la toma como mala.

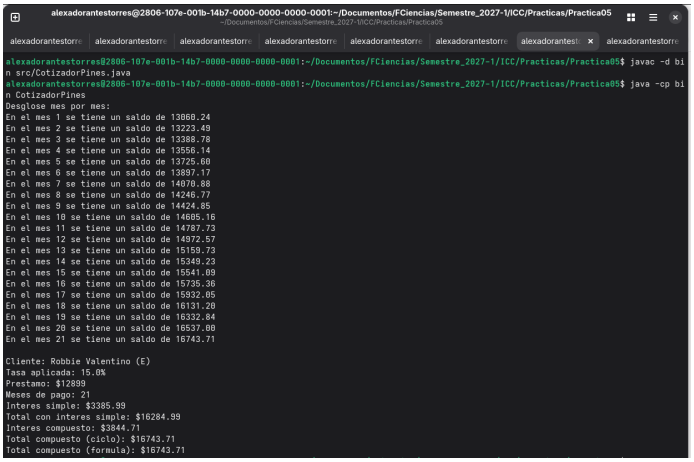


Figure 6: Ejecución char E

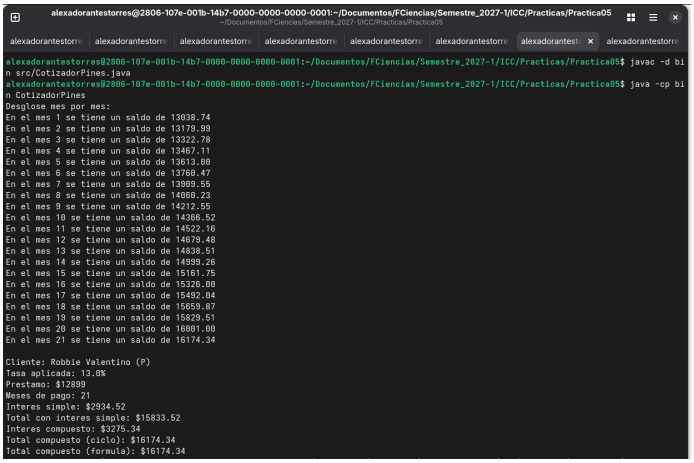


Figure 7: Ejecución char P

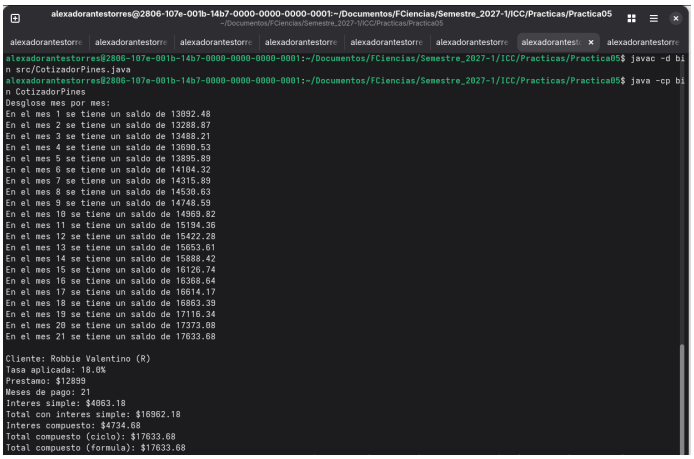


Figure 8: Ejecución char P

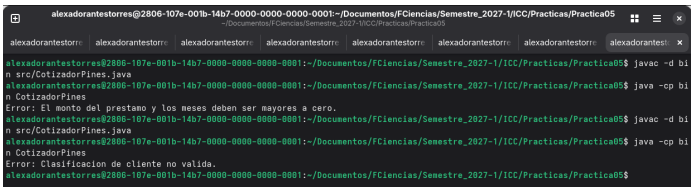


Figure 9: Errores, cambio variable a 0 y char e

Actividad 2.2 - Interés compuesto

En la Actividad 2.2, el desarrollo se enfocó en programar la simulación del interés compuesto y comprobar su precisión matemática paso a paso.

Para implementar la estrategia del ciclo for (punto 1), lo primero que hice fue adaptar la tasa anual a una tasa decimal mensual. Para esto, tomé la tasa del 15.0% y la dividí entre 1200, lo cual convierte el porcentaje a decimal y al mismo tiempo lo reparte entre los 12 meses del año. Después, guardé el saldo inicial en una variable asignándole el monto del préstamo de $12,899.00. Con esos valores listos, construí un ciclo for configurado para dar 21 vueltas, una por cada mes del plazo. En cada iteración, el programa multiplica el saldo del mes anterior por la expresión (1.0 + tasaMensual) para calcular el crecimiento de la deuda, guarda el nuevo valor e imprime inmediatamente en la terminal un renglón con el número de mes y el saldo acumulado. Esto permite visualizar en la pantalla exactamente cómo los intereses de cada mes se van sumando al capital para generar nuevos intereses en los periodos siguientes

Para la parte de la fórmula directa con Math.pow() (punto 2), calculé el saldo final elevando el factor (1.0 + tasaMensual) a la potencia de 21 meses con la función Math.pow() y multiplicándolo por el capital inicial de $12,899.00. Al comparar el resultado que arrojó esta fórmula directa contra el valor que quedó

guardado al terminar la última vuelta del ciclo for, ambos dieron exactamente la misma cantidad de $16,743.71, lo que confirmó que la lógica iterativa del ciclo funcionó a la perfección.

Finalmente, para la verificación con los datos de Robbie Valentino (punto 3), revisé que todas las salidas en la terminal coincidieran punto por punto con la tabla de referencia del PDF[cite: 10]. Al correr el programa con un préstamo de $12,899.00, un plazo de 21 meses y una tasa aplicada del 15.0%, se obtuvieron exactamente un interés simple de $3,385.99 (para un total con interés simple de $16,284.99) y un interés compuesto de $3,844.71, dando el total con interés compuesto de $16,743.71. Como todas las cifras cuadraron sin un solo centavo de diferencia, quedó demostrado que el cotizador realiza los cálculos financieros de forma totalmente correcta.

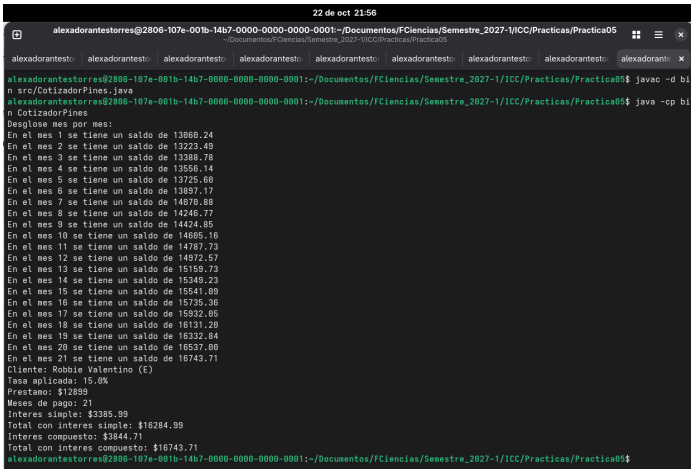


Figure 10: Datos Robbie correctos

Conclusión No entendí lo que hice, pero creo que está bien, la verdad me confundí en muchas partes, me equivoqué igual muchas veces y pido disculpas si llega a estar muy revuelto no fué mi intención, pero sentí que seguí aprendiendo mucho y medio entendí.